

-  GUÍA DE INTEGRACIÓN — YADFRIDEL
 -  ARCHIVOS QUE SON TU RESPONSABILIDAD
 -  ¿QUÉ HACE TU SERVICIO?
 -  PRE-REQUISITOS
 -  PASO A PASO COMPLETO
 - PASO 1 — Clonar el repositorio (solo si no lo tienes)
 - PASO 2 — Configurar identidad Git
 - PASO 3 — Asegurarte de estar en develop actualizada
 - PASO 4 — Crear tu rama de trabajo
 - PASO 5 — Compilar para verificar que funciona
 - PASO 6 — Agregar SOLO tu carpeta al staging
 - PASO 7 — Ver exactamente qué cambios incluyes
 - PASO 8 — Hacer commit
 - PASO 9 — Subir tu rama a GitHub
 - PASO 10 — Verificar en GitHub
 - PASO 11 — Crear el Pull Request
 -  SI NECESITAS ACTUALIZAR TU RAMA CON DEVELOP
 -  PROBLEMAS COMUNES Y SOLUCIONES
 - El servicio no compila — error de dependencia SendGrid
 - Error de conexión a Gmail API
 - "Port 8085 already in use"
 - "Updates were rejected" al hacer push
 - Accidentalmente agregué archivos de otro servicio al staging
 -  CHECKLIST PERSONAL — YADFRIDEL
 - Preparación
 - Antes del commit
 - PR
 -  INFORMACIÓN TÉCNICA DE TU SERVICIO

GUÍA DE INTEGRACIÓN — YADFRIDEL

Componentes a cargo: `notification-service/` **Rama de trabajo:** `feature/yadfridel-notification-service` **Ejecutar en:** FASE 4 (después de que Andrea haya mergeado user-service + gateway)

ARCHIVOS QUE SON TU RESPONSABILIDAD

```
notification-service/
├── src/
│   ├── main/
│   │   ├── java/com/eventos/notification_service/
│   │   │   ├── NotificationServiceApplication.java
│   │   │   ├── config/
│   │   │   │   └── AsyncConfig.java
│   │   │   ├── controller/
│   │   │   │   ├── NotificationController.java
│   │   │   │   └── HealthController.java
│   │   │   ├── dto/
│   │   │   │   ├── NotificationRequest.java
│   │   │   │   └── NotificationResponse.java
│   │   │   ├── email/
│   │   │   │   ├── EmailSender.java           ← interfaz base
│   │   │   │   ├── EmailTemplateBuilder.java ← modificado
│   │   │   │   ├── GmailApiEmailSender.java ← NUEVO
│   │   │   │   ├── NoOpEmailSender.java
│   │   │   │   ├── ResendEmailSender.java
│   │   │   │   ├── SendGridEmailSender.java
│   │   │   │   └── SmtplibEmailSender.java    ← modificado
│   │   │   ├── entity/
│   │   │   │   └── Notification.java
│   │   │   ├── repository/
│   │   │   │   └── NotificationRepository.java
│   │   │   ├── service/
│   │   │   │   ├── NotificationService.java
│   │   │   │   └── NotificationServiceImpl.java
│   │   └── resources/
│   │       └── application.properties        ← modificado
│   └── test/
├── .mvn/
├── pom.xml
└── mvnw / mvnw.cmd
```

 Los archivos marcados con ← **NUEVO** y ← **modificado** son los que tienen cambios recientes. Asegúrate de incluirlos todos al hacer el commit.

¿QUÉ HACE TU SERVICIO?

El **notification-service** es el encargado de enviar correos electrónicos a los usuarios del sistema. Soporta múltiples proveedores de email configurables:

Proveedor	Clase	Estado
Gmail API (OAuth2)	<code>GmailApiEmailSender.java</code>	✓ Activo por defecto
SMTP Gmail	<code>SmtplibEmailSender.java</code>	Disponible
SendGrid	<code>SendGridEmailSender.java</code>	Disponible
Resend	<code>ResendEmailSender.java</code>	Disponible
NoOp (sin envío)	<code>NoOpEmailSender.java</code>	Para pruebas

El proveedor activo se controla en `application.properties`:

```
app.mail.provider=mail-api
```



PRE-REQUISITOS

```
# Verificar Git instalado
git --version
# Resultado esperado: git version 2.x.x

# Verificar Java 17
java -version
# Resultado esperado: openjdk version "17.x.x"

# Verificar Maven Wrapper en notification-service
ls notification-service/mvnw
```



PASO A PASO COMPLETO

PASO 1 — Clonar el repositorio (solo si no lo tienes)

```
git clone [URL_DEL_REPO]
cd Eventos-Microservicios
```

PASO 2 — Configurar identidad Git

```
git config user.name "Yadfridel"  
git config user.email "tu-email@gmail.com"
```

PASO 3 — Asegurarte de estar en develop actualizada

⚠ **ESPERA** a que Rachel confirme que las Fases 1, 2 y 3 están mergeadas en **develop** (scripts ✓, auth-service ✓, user-service + gateway ✓)

```
git checkout develop  
git pull origin develop
```

Verifica que ves los commits anteriores:

```
git log --oneline -8  
# Debes ver commits de Francisco, Rachel y Andrea
```

PASO 4 — Crear tu rama de trabajo

```
git checkout -b feature/yadfridel-notification-service
```

Confirma:

```
git branch  
# * feature/yadfridel-notification-service
```

PASO 5 — Compilar para verificar que funciona

```
cd notification-service
./mvnw package -DskipTests
# En Windows sin bash: mvnw.cmd package -DskipTests
```

Resultado esperado:

```
[INFO] BUILD SUCCESS
[INFO] Total time: X:XX min
```

Volver a la raíz:

```
cd ..
```

PASO 6 — Agregar SOLO tu carpeta al staging

 **CRÍTICO:** Únicamente `notification-service/`. NADA más.

```
# Agregar toda la carpeta
git add notification-service/

# VERIFICAR inmediatamente
git status
```

Salida esperada:

```
On branch feature/yadfridel-notification-service
Changes to be committed:
  modified:   notification-
service/src/main/java/.../email/EmailTemplateBuilder.java
  modified:   notification-
service/src/main/java/.../email/SmtplibEmailSender.java
  new file:   notification-
service/src/main/java/.../email/GmailApiEmailSender.java
  modified:   notification-service/src/main/resources/application.properties
```

 Solo deben aparecer archivos dentro de `notification-service/`  Si aparece cualquier otra carpeta, quítala:

```
git restore --staged auth-service/  
git restore --staged user-service/  
git restore --staged gateway/  
git restore --staged event-service/  
git restore --staged inscrip-service/  
git restore --staged frontend-v2/
```

PASO 7 — Ver exactamente qué cambios incluyes

```
# Ver qué cambió en application.properties  
git diff --staged notification-service/src/main/resources/application.properties  
  
# Ver el nuevo archivo GmailApiEmailSender  
git diff --staged notification-  
service/src/main/java/com/eventos/notification_service/email/GmailApiEmailSender.ja  
va
```

PASO 8 — Hacer commit

```
git commit -m "feat(notification): notification-service con sistema multi-proveedor  
de email  
  
- Soporte multi-proveedor: Gmail API, SMTP, SendGrid, Resend, NoOp  
- GmailApiEmailSender.java (NUEVO): autenticación OAuth2 con Gmail API  
- SmtplibEmailSender.java: envío via SMTP Gmail configurado  
- EmailTemplateBuilder.java: plantillas HTML de notificaciones  
- NotificationController: endpoint POST /notifications/send  
- AsyncConfig: procesamiento asíncrono de emails  
- application.properties: proveedor activo gmail-api, configuración Railway MySQL  
(notification_db)"
```

PASO 9 — Subir tu rama a GitHub

Primera vez:

```
git push -u origin feature/yadfridel-notification-service
```

Cambios adicionales:

```
git push origin feature/yadfridel-notification-service
```

Force push cuando sea necesario (para sobrescribir):

```
# --force-with-lease es MÁS SEGURO que --force
# Solo falla si alguien más hizo push después de ti
git push origin feature/yadfridel-notification-service --force-with-lease
```

🚩 ¿Cuándo usar force push?

- Corregiste el último commit con `git commit --amend`
- Subiste algo incorrecto y necesitas reemplazarlo
- Reorganizaste commits con `git rebase`
- **NUNCA** en ramas `develop` o `main`

PASO 10 — Verificar en GitHub

1. Abre el repositorio en GitHub
2. Selecciona la rama `feature/yadfridel-notification-service`
3. Confirma que el árbol de archivos solo muestra cambios en `notification-service/`
4. Verifica archivos críticos:
 - `notification-service/src/main/resources/application.properties` — configuración de proveedores
 - `notification-service/src/main/java/.../email/GmailApiEmailSender.java` — archivo nuevo
 - `notification-service/src/main/java/.../email/EmailTemplateBuilder.java` — modificado

PASO 11 — Crear el Pull Request

Por GitHub UI:

1. GitHub mostrará un banner: **"Compare & pull request"** → clic

2. Configura:

- **Base:** `develop` ← ¡imprescindible!
- **Compare:** `feature/yadfridel-notification-service`
- **Título:** `feat(notification): notification-service con sistema multi-proveedor email`
- **Descripción:**

```
## notification-service

### Funcionalidades
- Sistema de notificaciones por email multi-proveedor
- Gmail API con OAuth2 (proveedor activo por defecto)
- SMTP Gmail como alternativa
- SendGrid y Resend disponibles
- Plantillas HTML de email (EmailTemplateBuilder)
- Procesamiento asíncrono (AsyncConfig)

### Archivos nuevos
- GmailApiEmailSender.java – implementación Gmail API OAuth2

### Archivos modificados
- SmtplibEmailSender.java
- EmailTemplateBuilder.java
- application.properties – configuración de proveedores y Railway MySQL

### Pruebas realizadas
- [ ] notification-service compila sin errores
- [ ] POST /notifications/send responde
- [ ] GET /notifications/health devuelve 200
- [ ] Email de prueba enviado exitosamente
```

3. Clic **"Create Pull Request"**

4. **Notifica a Rachel** que tu PR está listo

Por CLI (alternativa):

```
gh pr create \
  --base develop \
  --head feature/yadfridel-notification-service \
  --title "feat(notification): notification-service con sistema multi-proveedor email" \
  --body "Sistema de notificaciones email multi-proveedor: Gmail API (OAuth2), SMTP, SendGrid, Resend. Incluye GmailApiEmailSender nuevo y EmailTemplateBuilder actualizado."
```




SI NECESITAS ACTUALIZAR TU RAMA CON DEVELOP

Si entre que creaste la rama y el PR, alguien mergeó cambios a **develop**:

```
# 1. Actualizar develop local
git checkout develop
git pull origin develop

# 2. Volver a tu rama
git checkout feature/yadfridel-notification-service

# 3. Traer cambios de develop a tu rama
git merge develop

# 4. Si hay conflictos (muy improbable, tu carpeta es única)
# Git marcará los archivos – resuélvelos manualmente y:
git add notification-service/[archivo-resuelto]
git commit -m "resolve: conflictos con develop"

# 5. Push actualizado
git push origin feature/yadfridel-notification-service
```



PROBLEMAS COMUNES Y SOLUCIONES

El servicio no compila — error de dependencia SendGrid

```
cd notification-service
./mvnw dependency:resolve
# Luego intenta de nuevo:
./mvnw package -DskipTests
```

Error de conexión a Gmail API

El servicio está configurado con credenciales OAuth2 en `application.properties`:

```
gmail.client-id=863482226586-...  
gmail.client-secret=GOCS PX-...  
gmail.refresh-token=1//04jNr...
```

Si el refresh token expiró, el servicio usa `NoOpEmailSender` como fallback.

"Port 8085 already in use"

```
# En Windows  
netstat -ano | findstr :8085  
taskkill /PID [PID_NÚMERO] /F
```

"Updates were rejected" al hacer push

```
# Opción segura (si eres el único en tu rama)  
git push origin feature/yadfridel-notification-service --force-with-lease
```

Accidentalmente agregué archivos de otro servicio al staging

```
# Quitar sin perder los cambios locales  
git restore --staged auth-service/  
git restore --staged event-service/  
# etc.  
  
# Verificar que quedó limpio  
git status
```

Preparación

- ☐ Esperé confirmación de Rachel (Fases 1, 2 y 3 mergeadas en develop)
- ☐ `git checkout develop && git pull origin develop` ejecutado
- ☐ Veo commits de Francisco, Rachel y Andrea en `git log`
- ☐ Creé rama `feature/yadfridel-notification-service`

Antes del commit

- ☐ `notification-service/` compila: `./mvnw package -DskipTests` exitoso
- ☐ Solo `notification-service/` está en staging (`git status` verificado)
- ☐ `GmailApiEmailSender.java` incluido (archivo nuevo)
- ☐ `EmailTemplateBuilder.java` y `SmtplibEmailSender.java` incluidos (modificados)
- ☐ `application.properties` incluido
- ☐ Mensaje de commit descriptivo

PR

- ☐ Push exitoso a GitHub
- ☐ Verificado en GitHub: solo cambios en `notification-service/`
- ☐ **Base:** `develop` ✓ (NO `main`)
- ☐ PR creado
- ☐ Rachel notificada para revisión

INFORMACIÓN TÉCNICA DE TU SERVICIO

notification-service:

- Puerto: `8085`
- Base de datos: `notification_db` (Railway MySQL)
- Ruta en Gateway: `/notifications/**` → `http://localhost:8085`

Proveedores de email (configurable via `app.mail.provider`):

```
app.mail.provider=gmail-api    # Gmail API OAuth2 (default)
app.mail.provider=smtp        # SMTP Gmail
app.mail.provider=sendgrid    # SendGrid
app.mail.provider=resend      # Resend
app.mail.provider=noop        # Sin envío (pruebas)
```

Endpoints:

POST	/notifications/send	→ Enviar notificación/email
GET	/notifications	→ Listar notificaciones
GET	/notifications/{id}	→ Obtener notificación por ID
GET	/notifications/health	→ Health check

¿Quién llama a tu servicio?

- **inscrip-service** lo llama cuando un usuario se inscribe o cancela un evento
- Envía emails de confirmación automáticamente

Dependencias de tu servicio:

- Railway MySQL (**notification_db**)
- Gmail API / SMTP externo (para enviar emails)
- No depende de otros microservicios del proyecto